Haute école de gestion
Genève

Exploration des concepts et des techniques d'apprentissage par renforcement : De la théorie à la pratique

Travail de Master réalisé en vue de l'obtention du Master of Science HES-SO en Sciences de l'information par :

David Paulino

Conseillers au travail de Master :

Pr Alexandros KALOUSIS

Genève, le 31 mars 2026

Haute école de Gestion de Genève (HEG-GE)

Filière Information Science

Hes·so
Haute Ecole Spécialisée
de Suisse occidentale

1 Déclaration

Ce travail de Master est réalisé dans le cadre du Master en Sciences de l'information de la Haute école de gestion de Genève.

L'étudiant-e atteste que le travail rendu est le fruit de sa réflexion personnelle, a été rédigé de manière autonome sans avoir utilisé des sources autres que celles citées dans la bibliographie et a été vérifié par un logiciel de détection de plagiat.

L'étudiant-e accepte, le cas échéant, la clause de confidentialité.

L'utilisation des conclusions et recommandations formulées dans ce travail, sans préjuger de leur valeur, n'engage ni la responsabilité de l'étudiant-e, ni celle du-de la directeur-trice.

Fait à Genève, le 31 mars 2026

David Paulino

2 Remerciements

La partie où on remercie les gens qui nous ont aidé à réaliser ce travail de Master.

3 Résumé

Voilà le résumé de ce travail de Master. Il est important de bien résumer le travail pour que le lecteur puisse comprendre le contenu de ce travail sans avoir à le lire en entier.

Liste des tableaux

Table des figures

1 Boucle d'interaction agent-environnement en apprentissage par renforcement 10

2 Environnement Cliff Walking. 10

3 Itérations de l'évaluation de politique dans un monde de grille 4×4. Les états terminaux sont grisés. Inspiré de Silver (2015). 14

Table des matières

1 Déclaration **1**

2 Remerciements **2**

3 Résumé **3**

Liste des tableaux **4**

Table des figures **5**

4 Introduction **8**

5 État de l'art **9**

5.1 Le paradigme Agent-Environnement 9

5.1.1 Exemple : Cliff Walking 10

5.2 Spécificités et Concepts Clés 11

5.3 Processus Décisionnel de Markov (MDP) 12

5.4 Model-Based 13

5.4.1 Policy Evaluation et Policy Iteration 13

5.4.2 Value Iteration 14

5.5 Model-Free 15

5.5.1 Monte-Carlo 16

5.5.2 Temporal-Difference (TD) 17

5.6 Apprentissage en environnement inconnu 20

5.6.1 On-Policy vs Off-Policy 20
5.6.2 Le dilemme Exploration-Exploitation 20
5.6.3 SARSA 21
5.6.4 Q-Learning 21

6 Problématique **22**

7 Méthodologie **22**

8 Résultats **22**

9 Conclusion **22**

4 Introduction

Parmi les différents paradigmes de l'intelligence artificielle, l'apprentissage par renforcement (*Reinforcement Learning* ou RL) se distingue par sa capacité à former des agents décisionnels autonomes par le biais d'une interaction continue avec leur environnement (Sutton et al., 2015). Contrairement à l'apprentissage supervisé qui s'appuie sur des ensembles de données étiquetés, le RL repose fondamentalement sur un processus d'essais et d'erreurs. L'agent explore son environnement et affine sa *politique* d'action en se basant exclusivement sur des signaux de récompense, avec pour objectif ultime de maximiser ses gains cumulés sur le long terme.

Cette approche bio-inspirée a démontré des résultats spectaculaires dans des domaines aussi variés que la maîtrise de jeux stratégiques complexes (Zaffagni, 2022 ; *OpenAI Five*, 2018), le contrôle en robotique (Tang et al., 2024) ou encore la personnalisation des systèmes de recommandation (Iftikhar et al., 2024). Toutefois, son déploiement pose d'importants défis pratiques, notamment en ce qui concerne l'efficacité d'échantillonnage (*sample efficiency*), la formulation mathématique pertinente de la fonction de récompense, et la gestion du délicat compromis entre l'exploration de nouvelles actions et l'exploitation des connaissances déjà acquises.

5 État de l'art

Cette section introduit les fondations théoriques de l'apprentissage par renforcement, en s'appuyant principalement sur l'ouvrage de référence de Sutton et al. (2015) et les cours de Silver (2015).

5.1 Le paradigme Agent-Environnement

Le socle du RL est modélisé par une boucle d'interaction perpétuelle entre deux entités distinctes (voir Figure 1) :

— **L'agent** : le système apprenant et décisionnel.

— **L'environnement** : le système externe avec lequel l'agent interagit et qui réagit à ses actions.

À chaque pas de temps discret t, ce cycle se déroule comme suit :

1. L'agent perçoit une **observation** O_t de l'environnement et reçoit un signal scalaire de **récompense** R_t.
2. Sur la base de ces informations, l'agent sélectionne et exécute une **action** A_t dictée par sa stratégie (politique).
3. L'environnement intègre cette action, évolue vers un nouvel état, puis retourne à l'agent une nouvelle observation O_{t+1} ainsi qu'une récompense R_{t+1} évaluant la pertinence de la transition effectuée.

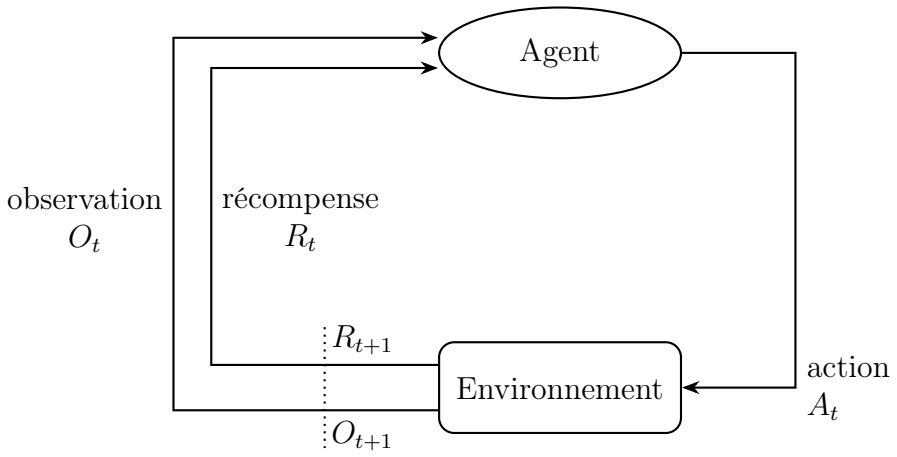

FIGURE 1 – Boucle d'interaction agent-environnement en apprentissage par renforcement

5.1.1 Exemple : Cliff Walking

L'environnement *Cliff Walking*[1] (Figure 2) illustre ces mécanismes. L'agent navigue sur une grille pour atteindre une cible en évitant une falaise. Il reçoit une récompense de -1 par pas parcouru (pour inciter à la rapidité) et de -100 avec retour au départ s'il tombe. Cet exemple met en évidence un compromis classique : l'agent doit équilibrer de la minimisation de la distance pénalisante tout en évitant un risque majeur.

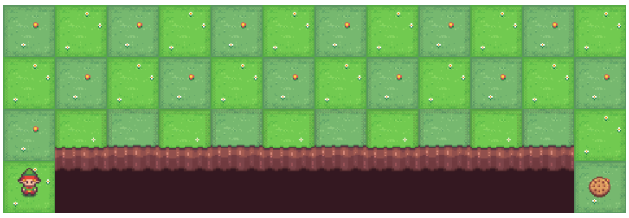

FIGURE 2 – Environnement Cliff Walking.

1. `https://gymnasium.farama.org/environments/toy_text/cliff_walking/`

5.2 Spécificités et Concepts Clés

Le RL se caractérise principalement par l'absence d'expert superviseur (aucun label ne vient indiquer la "bonne" action à prendre, seul le signal de récompense sert de retour), des *feedbacks* souvent différés soulignant le problème de l'assignation de crédit, et des données fortement corrélées temporellement (elles ne sont pas indépendantes et identiquement distribuées, ou i.i.d.).

Afin de formaliser ce problème, voici les cinq composantes fondamentales de la boucle d'interaction (Sutton et al., 2015) :

- **État de l'Environnement (S_t)** : L'état résume l'ensemble des informations de l'environnement pertinentes à un instant t pour que l'agent puisse déterminer son comportement. L'idéal est qu'il possède la *propriété de Markov* (c'est-à-dire qu'il englobe tout l'historique de manière compacte).
- **Action (A_t)** : La décision exécutée par l'agent dans l'état S_t. Selon le problème, l'espace des actions peut être discret (ex : aller en haut, en bas, à gauche, à droite) ou au contraire continu (ex : le degré de braquage d'un volant).
- **Récompense (R_{t+1})** : Le signal scalaire émis par l'environnement qui évalue purement et immédiatement le succès de l'action A_t. L'objectif central du RL est d'optimiser l'accumulation temporelle de ces retours mesurables.
- **Politique (π)** : La stratégie propre à l'agent. Cette fonction détermine quelles actions entreprendre en fonction de chaque état observé. Elle peut être déterministe ($a = \pi(s)$) ou stochastique, c'est-à-dire attribuer une probabilité donnée d'exécuter une action ($P(A_t = a|S_t = s)$).
- **Fonction de valeur (V^π, Q^π)** : Mesure la qualité ("value") d'un état (ou d'une paire état-action), c'est-à-dire l'espérance mathématique des récompenses qui seront cumulées à long terme à partir de cet état, en suivant la politique π.

5.3 Processus Décisionnel de Markov (MDP)

L'environnement d'apprentissage dans sa globalité est formellement modélisé par un Processus Décisionnel de Markov (MDP). Ce puissant cadre mathématique repose sur la **propriété de Markov** : l'état actuel S_t contient toutes les informations nécessaires et suffisantes pour extrapoler le comportement futur. La prédiction de l'état suivant dépend ainsi uniquement de l'état présent, indépendamment de toute l'historique de navigation passée ($P(S_{t+1}|S_t) = P(S_{t+1}|H_t)$).

Un MDP se décrit classiquement par le système $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$ et $\mathcal{A}$ représentent respectivement l'ensemble des états que peut prendre l'environnement et l'ensemble des actions appartenant à l'agent.
- $\mathcal{P}^a_{ss'} = \mathbb{P}[S_{t+1} = s'|S_t = s, A_t = a]$ définit la matrice des probabilités de transition. Elle modélise la nature (déterministe ou probabiliste) des changements d'état résultant d'une action a spécifique.
- $\mathcal{R}^a_s = \mathbb{E}[R_{t+1}|S_t = s, A_t = a]$ fixe la récompense espérée en transitant par l'action a à partir de l'état s.
- $\gamma \in [0, 1]$ est le facteur d'actualisation ou de dépréciation (*discount factor*). Il quantifie l'intérêt lié à l'horizon temporel des récompenses : une valeur de γ proche de 0 réduit le temps de vue et favorise les gains très immédiats, tandis qu'une valeur se rapprochant de 1 insiste sur la conception de stratégies à long terme.

L'agent vise in fine à maximiser son **retour cumulé** global G_t, exprimé comme une série géométrique de ces récompenses actualisées :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{1}$$

Pour l'exemple du labyrinthe, on pourrait imaginer une récompense $\mathcal{R}$ de -1 par étape de déplacement (pour pénaliser le temps perdu) et de +10 en trouvant la sortie, chaque caséte constituant un état $\mathcal{S}$, et les pas (G, D, H, B) constituant les actions $\mathcal{A}$. Le facteur γ garantit alors que l'agent privilégiera mathématiquement un chemin court et rapide vers la sortie plutôt qu'une errance prolongée.

5.4 Model-Based

Pour classifier les méthodes d'apprentissage par renforcement, il est utile de distinguer les approches basées sur un modèle (*model-based*) de celles sans modèle (*model-free*). Les méthodes *model-based* supposent que l'agent possède une connaissance complète de la dynamique de l'environnement, représentée par les fonctions de transition $\mathcal{P}$ et de récompense $\mathcal{R}$.

Grâce à cette connaissance, l'agent peut planifier ses actions en évaluant mentalement leurs conséquences avant même de les exécuter. Ainsi, plutôt que d'apprendre par essais et erreurs, l'agent calcule mathématiquement la stratégie optimale dès le départ, pour autant que la complexité du problème le permette.

5.4.1 Policy Evaluation et Policy Iteration

La *Policy Iteration* (ou itération de politique) est un processus qui permet de trouver la politique optimale en alternant deux étapes clés. La première consiste à évaluer la politique actuelle pour connaître sa valeur exacte (*Policy Evaluation*). La seconde étape cherche ensuite à l'améliorer en choisissant systématiquement la meilleure action possible en fonction des résultats de l'évaluation précédente de manière *greedy* (*Policy Improvement*).

Ce processus d'évaluation suivi d'une amélioration est répété itérativement. La convergence vers la politique qualifiée d'optimale (π^*) garantit alors systématiquement $V^{\pi^*} \geq V^{\pi}$ quel que soit l'état initial comparé (Silver, 2015).

Mesurer la performance d'une politique donnée équivaut à quantifier l'espérance mathématique cumulative des récompenses reçues si l'agent respecte continuellement π. Pour ce faire, nous employons une récurrence itérative construite à partir d'une estimation à zéro ($V_0(s) = 0$) :

$$V_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_k(s') \right] \tag{2}$$

Au fil des itérations, les récompenses associées aux actions se propagent

d'état en état (comme l'illustre la Figure 3, où l'influence des états terminaux rayonne vers les cases adjacentes à chaque étape k). Sous réserve d'un nombre infini d'itérations, les calculs convergent et s'affinent.

0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0

$k = 0$

$\longrightarrow$

0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0

$k = 1$

$\longrightarrow$

0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0

$k = 2$

FIGURE 3 – Itérations de l'évaluation de politique dans un monde de grille 4×4. Les états terminaux sont grisés. Inspiré de Silver (2015).

Bien qu'une évaluation parfaite soit mathématiquement faisable, l'objectif fondamental demeure l'identification rapide d'une stratégie de jeu plus favorable. Dans les faits, un nombre restreint d'itérations k d'évaluation confère une estimation suffisamment robuste pour orienter la décision opportuniste *greedy*.

La mise à jour de la politique s'oriente alors selon le critère de sélection de l'action optimale :

$$\pi'(s) = \arg\max_{a \in \mathcal{A}} q_\pi(s, a) \tag{3}$$

Tant que la nouvelle politique générée diffère, ces opérations inter-cycles se relaient. Ce processus prend le nom d'**itération modifiée** lorsque l'évaluation s'interrompt pour économiser du temps de calcul dès lors qu'un seuil minime ϵ est atteint entre deux évaluations.

5.4.2 Value Iteration

Contrairement à l'itération de politique qui évalue une stratégie précise à chaque étape, la *Value Iteration* s'attaque à l'optimisation par la fusion des

deux étapes précédentes en une seule opération globale. L'algorithme cherche à trouver directement la valeur maximale de chaque état, sans avoir besoin de stocker une politique intermédiaire lors de l'entraînement.

Cette mécanique réside exclusivement dans l'introduction d'un opérateur "maximum" qui condense l'évaluation et l'amélioration :

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_k(s') \right] \tag{4}$$

Pour chaque état s et à chaque étape, l'agent met à jour ses valeurs en supposant qu'il prendra systématiquement l'action future la plus rémunératrice. En pratique, on arrête les calculs (la phase de convergence) dès que la propagation des valeurs devient marginale d'une itération à l'autre (par exemple lorsque la modification $\max_s |V_{k+1}(s) - V_k(s)| < \epsilon$).

Une fois que les valeurs de l'ensemble V sont stabilisées, l'approche pour définir de quel comportement l'agent doit faire preuve (extraire π^*) se fait intuitivement. Il suffit à l'agent de regarder les états accessibles depuis sa position actuelle et de choisir l'action lui rapportant la valeur la plus élevée :

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}} \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V^*(s') \right] \tag{5}$$

5.5 Model-Free

Cependant, l'utilisation de l'apprentissage par renforcement s'applique majoritairement à des environnements réels complexes où les règles du monde sont stochastiques, inconnues, ou trop volumineuses pour être modélisées. Sans la connaissance des formules de probabilité de transition $\mathcal{P}$ ni des fonctions de récompense $\mathcal{R}$, les méthodes *model-based* s'avèrent inapplicables car l'agent ne peut pas se projeter mathématiquement.

Pour surmonter cet obstacle, les méthodes dites *model-free* (sans modèle) effectuent un apprentissage fondé sur l'expérience. Elles s'alimentent exclusivement des données rencontrées au cours de l'interaction de l'agent avec

l'environnement pour estimer peu à peu la valeur des actions. Ce changement d'approche correspond à l'idée exprimée par Silver (2015) caractérisant ces processus comme une étape de véritable apprentissage par expérience (*learning*) plutôt que de simple planification en amont (*planning*).

5.5.1 Monte-Carlo

L'approche de Monte-Carlo (MC) s'appuie sur une idée très intuitive en ce qui concerne l'apprentissage basé sur l'expérience : la valeur d'un état correspond simplement à la moyenne de la somme des récompenses accumulées lors des récents passages par cet état. À l'inverse des méthodes *model-based* qui mettent à jour les estimations en s'appuyant sur d'autres estimations de valeurs (créant ainsi une chaîne de prédictions imbriquées), Monte-Carlo se fie uniquement aux résultats complets observés directement sans dépendre d'approximations intermédiaires.

Le fonctionnement de base des méthodes de Monte-Carlo repose sur trois prérequis. Le premier est la nécessité de générer un apprentissage par une séquence d'expériences (états, actions, et récompenses) accumulée par l'agent au cours de ses essais successifs. Par corrélation, l'environnement étudié doit obligatoirement être de nature "épisodique". C'est-à-dire qu'une interaction doit toujours avoir une fin déterminée, au contraire de processus cycliques tournant infiniment. En effet, tout le principe repose sur le fait de pouvoir calculer le retour accumulé et total de la session, ce qui requiert d'atteindre la fin de l'épisode (au *game over* d'une partie d'échecs par exemple) pour être possible. Naturellement, ces données servent une estimation qui, répétée pour chaque nouvel essai, s'affine statistiquement au fil de l'augmentation du nombre de lancers.

Pour des raisons d'efficacité, l'algorithme ne garde pas en mémoire l'intégralité des résultats passés pour calculer sa moyenne de zéro à chaque fois. Il utilise plutôt une méthode de mise à jour incrémentale. À la fin de chaque épisode, il passe en revue les états qu'il a visités et corrige directement sa dernière estimation :

$$V(S_t) \leftarrow V(S_t) + \alpha \left(G_t - V(S_t)\right) \tag{6}$$

Ici, la différence $(G_t - V(S_t))$ exprime très exactement l'erreur mesurée entre le résultat du retour nouvellement calculé en finissant l'épisode (G_t) et l'estimation jusqu'alors théorisée pour l'état en passe d'être mis à jour $(V(S_t))$. Cette erreur est tempérée par un coefficient nommé α (*learning rate*), ce qui évite un bouleversement complet des connaissances en cas de coup de chance et lisse l'apprentissage.

Au cours d'un épisode, un état précis peut faire l'objet de visites répétées (comme l'agent passant par une case du labyrinthe plusieurs fois de suite). Ce détail force l'approche de l'algorithme à privilégier deux méthodes de décompte disctintes. Avec le premier traitement (*First-Visit*), l'algorithme relève seulement et ajuste la valeur basée sur ce qui s'est déroulé après avoir touché l'état la toute première fois au cours d'une occurrence. La méthode diamétralement opposée (*Every-Visit*) reconsidère l'évaluation de l'état pour chacune de ses apparitions.

Si cette méthode présente l'avantage d'être très robuste (ses estimations sont toujours basées sur des résultats réels et ne sont donc pas perturbées par de potentielles prédictions erronées sur d'autres états), elle souffre cependant de défauts majeurs dans son rythme d'apprentissage. En effet, l'agent doit systématiquement attendre la fin définitive de l'épisode pour mettre à jour ses connaissances, ce qui rend la progression particulièrement lente selon l'environnement. De plus, cela crée un problème critique de forte variance : un coup de chance exceptionnel ou un désastre va impacter le score final de tout l'épisode. Cela peut conduire à des mises à jour très versatiles et irrégulières, rendant l'apprentissage instable et difficile à maîtriser.

5.5.2 Temporal-Difference (TD)

Les méthodes des différences temporelles (TD) combinent les forces des méthodes de Monte-Carlo et de la programmation dynamique. À l'instar de Monte-Carlo, elles apprennent directement à partir de l'expérience brute, sans nécessiter de modèle de transition $(\mathcal{P})$. Cependant, à l'instar de la programmation dynamique, elles s'affranchissent de l'attente de la fin de l'épisode en utilisant le *bootstrapping*[2] (Sutton et al., 2015).

2. Principe consistant à mettre à jour l'estimation d'un état en se basant sur l'estimation de l'état suivant, plutôt que sur le retour final réel.

Pour illustrer cette nuance, Silver (2015) propose l'analogie du trajet en voiture. Imaginons un conducteur estimant arriver chez lui à 18h30. À 18h15, il se retrouve bloqué dans un embouteillage. Une approche Monte-Carlo obligerait le conducteur à attendre d'être effectivement garé chez lui pour constater son retard et ajuster ses prévisions futures. À l'inverse, l'approche TD permet au conducteur de "bootstrapper" : dès qu'il voit le bouchon, il actualise instantanément son estimation ("le trajet prendra finalement 50 minutes") sans avoir besoin de terminer sa route.

Mathématiquement, le mécanisme le plus simple, le TD(0), remplace le gain réel G_t par une **cible estimée** (*TD target*) composée de la récompense immédiate et de la valeur de l'état suivant :

$$V(S_t) \leftarrow V(S_t) + \alpha\big(\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)\big) \tag{7}$$

La différence entre cette cible et l'estimation actuelle est nommée **erreur TD** ($\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$). Contrairement à Monte-Carlo, le TD(0) peut fonctionner dans des environnements continus ou cycliques car il apprend à chaque pas.

Par nature, le TD(0) ne regarde qu'un seul pas dans le futur avant de consulter son estimation. On peut toutefois généraliser ce concept en enregistrant n étapes de récompenses réelles avant de "bootstrapper" sur l'état S_{t+n}. On obtient alors le **retour à n pas** (*n-step return*) :

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}) \tag{8}$$

Si $n = 1$, l'approche équivaut au TD(0). Si $n \to \infty$, elle rejoint la méthode de Monte-Carlo. Le choix de n devient alors un curseur permettant de modifier l'horizon de prédiction de l'agent.

Cependant, plutôt que de choisir arbitrairement ce nombre de pas n, l'algorithme $TD(\lambda)$ propose une synthèse en unifiant simultanément tous les horizons possibles ($n = 1, 2, \ldots, \infty$). Ce mélange est réalisé via une moyenne pondérée par un paramètre $\lambda \in [0, 1]$, définissant ainsi le **λ-return** (G_t^λ) :

$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}G_t^{(n)} \tag{9}$$

Bien que le G_t^λ offre une cible robuste, il souffre du même défaut que Monte-Carlo : il regarde vers le futur et nécessite d'attendre la fin de l'épisode pour être calculé (approche dite *Forward View*). Pour rendre l'apprentissage possible en temps réel, on adopte une approche *Backward View* s'appuyant sur les **traces d'éligibilité** $E_t(s)$ (Silver, 2015).

L'équivalence mathématique entre ces deux visions permet de ne plus utiliser directement le λ-return dans la mise à jour, mais de propager l'erreur TD locale (δ_t) vers le passé. Chaque état visité laisse une "empreinte" qui s'estompe avec le temps selon le facteur λ [3]. La mise à jour de la trace d'éligibilité pour chaque état s à chaque pas de temps t s'écrit ainsi :

$$E_t(s) = \gamma\lambda E_{t-1}(s) + 1(S_t = s) \tag{10}$$

Ainsi, à chaque pas de temps, la fonction de valeur de **tous les états** est ajustée proportionnellement à leur responsabilité dans l'erreur courante :

$$V(s) \leftarrow V(s) + \alpha\delta_t E_t(s) \tag{11}$$

Cette mécanique est particulièrement efficace : dans un jeu vidéo, si un agent saute sur plusieurs plateformes avant de tomber dans un piège, le $TD(\lambda)$ permettra de baisser instantanément la valeur de toutes les plateformes de la séquence (beaucoup pour la dernière, un peu moins pour les précédentes) au prorata de leur responsabilité dans l'issue finale.

3. Le paramètre λ contrôle la persistance de la trace : une valeur proche de 0 restreint la mise à jour aux états les plus immédiats (approche "myope"), tandis qu'une valeur proche de 1 maintient l'empreinte des états anciens plus longtemps.

5.6 Apprentissage en environnement inconnu

Le passage de l'évaluation à l'optimisation consiste à améliorer la politique de l'agent sans connaissance préalable de la dynamique de l'environnement. Dans ce contexte *model-free*, l'utilisation de la fonction de valeur d'état $V(s)$ s'avère insuffisante : pour choisir l'action optimale, l'agent aurait besoin de connaître les probabilités de transition $\mathcal{P}$. Par conséquent, les algorithmes de contrôle se basent sur la fonction action-valeur $Q(s, a)$, permettant d'identifier directement l'action maximisant le gain attendu (Silver, 2015).

5.6.1 On-Policy vs Off-Policy

Deux paradigmes fondamentaux s'opposent dans l'apprentissage sans modèle :

- **On-Policy** : L'idée est d'apprendre "sur le tas". L'agent évalue et optimise la politique π qu'il est en train d'exécuter. Les expériences utilisées pour la mise à jour reflètent directement ses propres décisions, incluant ses tentatives d'exploration.
- **Off-Policy** : L'idée est d'apprendre en observant une autre politique. L'agent procède à l'optimisation d'une politique cible π (généralement la politique optimale) à partir d'expériences générées par une politique de comportement μ distincte. Cela permet de séparer l'exploration de l'évaluation finale (Sutton et al., 2015).

5.6.2 Le dilemme Exploration-Exploitation

Pour garantir la convergence vers une solution idéale, l'agent ne peut se contenter d'être *greedy* dès le départ, au risque de rester piégé dans un optimum local. Il doit arbitrer entre l'exploitation de ses connaissances actuelles (issues de l'évaluation) et l'exploration de nouvelles stratégies. La méthode la plus répandue est la stratégie ϵ**-greedy** :

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{m} & \text{si } a = \text{argmax}_{a \in \mathcal{A}} Q(s, a) \\ \frac{\epsilon}{m} & \text{sinon} \end{cases} \tag{12}$$

En pratique, on utilise un *scheduler* pour réduire progressivement la valeur de ϵ au fil du temps. Cette décroissance permet de satisfaire les conditions **GLIE**[4] : l'agent explore massivement au début, puis stabilise son comportement vers une politique déterministe optimale une fois l'espace état-action suffisamment parcouru. Cependant, un *epsilon* qui descendrait trop vite à 0 créerait un manque d'exploration, empêchant l'agent de découvrir l'intégralité des récompenses et biaisant l'optimisation.

5.6.3 SARSA

L'algorithme SARSA est une méthode d'optimisation **On-Policy**. Il met à jour la fonction Q en utilisant la récompense immédiate et l'évaluation de l'action suivante (A') effectivement choisie par la politique actuelle de l'agent. Son nom illustre d'ailleurs la séquence (S, A, R, S', A') utilisée pour la mise à jour :

$$Q(S, A) \leftarrow Q(S, A) + \alpha\big(R + \gamma Q(S', A') - Q(S, A)\big) \tag{13}$$

Parce qu'il inclut l'action réelle A' de l'agent (et donc les erreurs potentielles liées à l'exploration) dans son calcul, SARSA réalise une optimisation "prudente". Il évalue les zones de l'environnement en tenant compte du fait qu'un mouvement aléatoire dû à ϵ pourrait entraîner une chute brutale de la récompense (Silver, 2015).

5.6.4 Q-Learning

À l'inverse, le Q-Learning est une méthode d'optimisation **Off-Policy**. Il s'affranchit de l'action réellement choisie par l'agent pour la mise à jour en utilisant l'opérateur *maximum*. Il optimise directement la valeur de la politique optimale, indépendamment du comportement exploratoire :

$$Q(S, A) \leftarrow Q(S, A) + \alpha\big(R + \gamma \max_{a'} Q(S', a') - Q(S, A)\big) \tag{14}$$

Ici, par rapport à SARSA, la mise à jour se base sur l'évaluation de la meilleure action possible (max), sans tenir compte de l'action A_{t+1} réelle-

4. *Greedy in the Limit with Infinite Exploration*

ment effectuée par l'agent. Cette approche permet de converger vers la fonction d'optimalité de Bellman et de converger vers la politique optimale même si l'agent continue d'explorer activement son environnement.

6 Problématique

7 Méthodologie

8 Résultats

9 Conclusion

Références

IFTIKHAR, Arta ; GHAZANFAR, Mustansar Ali ; AYUB, Mubbashir ; ALI ALAHMARI, Saad ; QAZI, Nadeem ; WALL, Julie, 2024. A Reinforcement Learning Recommender System Using Bi-Clustering and Markov Decision Process. *Expert Systems with Applications* [en ligne]. T. 237, p. 121541 [visité le 2026-02-23]. ISSN 0957-4174. Disp. à l'adr. DOI : 10.1016/j.eswa.2023.121541.

OpenAI Five, 2018 [en ligne]. 2018-06-25. [visité le 2026-02-23]. Disp. à l'adr.: https://openai.com/index/openai-five/.

SILVER, David, 2015. *Lectures on Reinforcement Learning*.

SUTTON, Richard S; BARTO, Andrew G, 2015. Reinforcement Learning: An Introduction.

TANG, Chen ; ABBATEMATTEO, Ben ; HU, Jiaheng ; CHANDRA, Rohan ; MARTÍN-MARTÍN, Roberto ; STONE, Peter, 2024. *Deep Reinforcement Learning for Robotics : A Survey of Real-World Successes* [en ligne]. 2024-09-16. [visité le 2026-02-23]. Disp. à l'adr. DOI : 10.48550/arXiv.2408.03539.

ZAFFAGNI, Marc, 2022. *Définition | AlphaGo : qu'est-ce que c'est ? | Futura tech* [en ligne]. Futura, 2022-12-01. [visité le 2026-02-23]. Disp. à l'adr. : `https://www.futura-sciences.com/tech/definitions/intelligence-artificielle-alphago-16467/`.